

Software Design Document
SP-14 - Green - Chess AI
CS 4850 - Section 03 – Fall 2025
Sep 10, 2025

			
Matt Staats Developer	Jason Nguyen Documentation	Carlos Pena Documentation	Chiagoziem Iteogu Developer

Project Team

Roles	Name	Team Function	Contact (Cell Phone) and Alt Email
Project owner or Advisor	Sharon Perry	Facilitate project progress; advise on project planning and management.	770.329.3895 Sperry46@kennesaw.edu
Team leader	Matt Staats	Team Leader & Developer	404.405.6710 matthewcstaats@gmail.com
Team members	Jason Nguyen	Documentation & Testing	404.747.3352 Neilgroup101@gmail.com
	Carlos Pena	Documentation & Website	404.519.5636 Cromero2511@gmail.com
	Chiagoziem Iteogu	Developer & Testing	478.955.1662 Goziem.iteogu@yahoo.com

Table of Contents

1. Introduction and Overview.....	
1.2.1 Introduction.....	
1.2.2 System Overview.....	
2. Design Considerations.....	
2.1 Assumptions and Dependencies.....	
2.2 General Constraints.....	
2.3 Development Methods.....	
3. Architectural Strategies.....	
3.1 System Architecture.....	
3.2 Input and Output Model.....	
3.3 Testing and Quality Assurance.....	
3.4 Development Environment and Version Control.....	
4. System Architecture.....	
4.1 Overview.....	
4.2 Subsystem Architecture.....	
4.2.1 Frontend (React).....	
4.2.2 Game Logic / AI Engine.....	
4.2.3 Data / API Layer.....	
4.2.4 Integration and Interaction.....	
4.2.5 Rationale and Trade-offs.....	
5. Detailed System Design.....	
5.1 Classification.....	
5.2 Definition.....	
5.3 Constraints.....	
5.4 Resources.....	
5.5 Interface/Exports.....	
6. Glossary.....	
7. Bibliography.....	
8. UML Diagram of System Architecture	

1. Introduction and Overview

1.2.1 Introduction

This document describes the design for the AI Chess game. It goes through an outline that shows the system's architecture, design considerations and detailed specifications for the parts that make up the AI chess game. The purpose of the design document is to provide a roadmap that will help with the development and testing of this software.

1.2.2 System Overview

- Rules Engine: Chess.js makes sure all moves follow all official chess rules.
- Data layer: In JSON format the moves and game states are stored
- Ai Engine: Uses minimax to evaluate and select moves.
- game director: coordinates rules, AI and database updates.
- Web Interface: Displays the board in chessboard.js and manages the front-end user interaction through React.

2. Design Considerations

2.1 Assumptions and Dependencies

- The chess game backend will be developed in Python.
- The system will run in a web browser using React to build the UI
- chess.js will handle all the rule validation and the game state management
- Chessboard.js will render the interactive chessboard
- All game data which include moves, states, metadata will be stored in a JSON based database (optional)

2.2 General Constraints

- Platform must run in a desktop or laptop web browser (Chrome, Firefox, Edge, and Safari with proper testing)
- No mobile or tablet devices
- Performance: Ai must compute moves fast enough for smooth gameplay.
- No cloud deployment, multiplayer head-to-head limited to locally only.
- No AI difficulty scaling. AI will make the best optimal choice based on our custom minimax algorithm

- No save and load game functionality. The game must be played and finished in one session.
- Game data will be stored in JSON format

2.3 Development Methods

The chess system will be developed under the Agile framework, combined with CI/CD (Continuous Integration and Continuous Deployment) practices. The Agile framework allows the team to work iteratively and incrementally, breaking the project into separate sprints that focus on specific features such as the frontend chessboard, AI engine functionality, or game logic processing. Continuous feedback from the Agile process provides flexibility for adapting to changing requirements and incorporating validation from user or client input.

To support Agile development, CI/CD practices are incorporated to maintain high code quality throughout the project. Every code change pushed to the repository is built, compiled, tested, and validated before merging into the main branch. This ensures that errors are caught early, reduces the cost and delay associated with unexpected bugs, and preserves overall code quality. The combination of Agile and CI/CD practices enhances the efficiency, reliability, and scalability of the system, ensuring that future releases can be delivered smoothly and effectively.

- Modular coding: Encapsulates each responsibility into its own class or class.
- Scenario Based testing: test all valid conditions through test games.

3. Architectural Strategies

3.1 System Architecture

The architecture of the system implements strategies of building upon existing software libraries for the development of web-based applications. The frontend utilizes React.js for our user interface, with Chessboard.js for gameboard visualization. The backend stores the game state, logic, and AI engine in Python, using the Flask API to communicate with the frontend and update the chessboard for real-time gameplay against the AI.

This strategy helps with maintainability and scalability by separating the two components, ensuring modularity when developing game logic, AI processing, and the user interface. The clear separation of concerns between the user interface and game logic allows the frontend and backend to operate independently without affecting existing dependencies, enabling scalability to add additional components or features in the future. Other considerations, such as building the system in a single unified language or as a desktop

application, would limit flexibility for future enhancements, including web-based multiplayer support and cross-platform compatibility.

3.2 Input and Output Model

The system's input and output model is defined as follows: the user provides input by dragging and dropping a valid chess move, and the output is an updated game state with the new chessboard rendering. The format follows PGN stored in JSON files, which communicates with the Flask API to update the backend game state, as well as the frontend chessboard. The AI engine then evaluates the game state, generates a move, and updates the game state, which is relayed through the API to update the user interface and reflect the new game state.

3.3 Testing and Quality Assurance

For testing purposes, the system will implement a continuous integration (CI) pipeline to ensure that every build is compiled, validated, and tested before code is pushed to the main branch. This approach allows for early bug detection, smoother integration across components, and the preservation of overall code quality.

Unit testing will validate individual functions and modules, helping to identify data bottlenecks or logic errors at the component level. Black Box testing will ensure that the system behaves correctly from a user perspective, confirming that the application meets functional requirements and delivers a reliable user experience. Integration testing will focus on the communication between the frontend (React.js with Chessboard.js) and the backend (Flask API with Python-based game logic), while end-to-end testing will verify that the entire workflow operates seamlessly. These layered testing approaches enhance maintainability and scalability, ensuring the system remains reliable as new features and components are introduced.

3.4 Development Environment and Version Control

To maintain accessibility and maintainability of the source code, we use GitHub for version control and file sharing throughout the development process. By creating an organizational account, all members have permission to contribute to the codebase, ensuring that the project remains up to date, supports efficient collaboration, and provides a structured environment for maintenance and organization. Our preferred IDE is Visual Studio Code, chosen for its versatility and ease of use in web development. Its wide range of extensions, built-in Git integration, and support for multiple programming languages make it well-suited for managing both the frontend and backend components of the project.

4. System Architecture

This section presents a high-level architectural overview of the Chess AI Game Website and the logical partitioning of its responsibilities among subsystems and major components. The design supports modularity, agile workflows, and ease of modification, in line with a scenario-based approach.

Functional Partitioning

- **Frontend:** The user interface is built with React. All interactive elements, game state presentations, and user input handling occur on this layer. The frontend communicates with the backend using JSON over HTTP (WebSocket for real-time updates), and employs chess.js to execute move validation, legal move generation, and lightweight local game state management.
- **Backend:** Implemented in Python, the backend provides core application logic, game management, persistent storage, AI computation, and orchestrates communication among modules. The backend exposes a REST API, returning and accepting JSON payloads for all data exchange.
- **AI Engine:** A modular component within the backend, implemented in Python, responsible for computing moves using configurable AI logic. This component is designed for extensibility and scenario-based testing (swapping algorithms or difficulty levels).
- **Storage/Database Subsystem:** Responsible for saving user data, game histories, and scenario configurations. For local operation, this may be a lightweight file-based solution (JSON files).
- **Scenario Module:** Encapsulates scenario-based workflows, such as tutorials, puzzles, and analysis. This module leverages the same move-processing infrastructure as regular gameplay but loads and manages custom board states and solution paths.
- **Integration Layer:** Handles communication protocols and data serialization/deserialization between frontend, backend, and scenario modules, ensuring all interactions happen via strict JSON schemas.

Assigning Responsibilities

- The Frontend provides user-facing components but delegates all persistent logic, state reconciliation, and AI processing to the backend.
- The Backend coordinates state, handles API requests, persists data, and invokes AI or scenario modules as needed.
- The Scenario Module enables swapping or customizing modules for different chess exercises without altering core backend logic, promoting an agile and modular design philosophy.

Architectural Decisions and Rationale

- **Modular Coding:** System components are implemented with well-defined interfaces, allowing independent development, testing, and substitution (swap AI backends or change the storage mechanism).
- **Agile Flow:** Rapid prototyping and iterative updates are supported by scenario encapsulation and API boundaries.
- **Scenario-Based Building:** Scenario configurations are managed as data objects (JSON), allow quick extension or creation of new chess situations without backend rewrites.
- **Local Hosting:** All components are designed to run locally, requiring minimal to no external dependencies beyond Python, Node.js, and the chosen database/file system.

4.1 Overview

The system is partitioned into three main subsystems:

- Frontend (React)
- Game Logic Engine (Python & chess.js)
- Data Exchange/API Layer (JSON over HTTP/WebSocket)

4.2 Subsystem Architecture

Each subsystem is independently developed and tested, allowing for modular upgrade and scenario-based extensions such as puzzles, analysis, or new AI difficulty settings. The architecture supports agile workflows, with clear API boundaries enabling parallel tasking for frontend and backend teams.

4.2.1 Frontend (React)

- Implements the user interface, including the chessboard, controls, move history, and settings.
- Communicates with backend APIs via asynchronous JSON requests
- Receives real-time game updates (moves, AI responses, game state) through HTTP or Web Sockets.
- Utilizes chess.js locally for immediate client-side move validation, allowing for smooth UI feedback and optimistic updates.
- Modular components (board, move list, chat, scenario selector) encourage reuse and isolated testing.

4.2.2 Game Logic/ AI Engine (Python and chess.js)

- Python provides core AI logic (move calculation, difficulty modes, scenario-aware play, logging).
- chess.js is used for legal move validation, FEN parsing, and core chess rules enforcement.
- Scenario-based build tools (Python-driven or config-file-based) inject custom game setups, AI constraints, or puzzle definitions via JSON.
- RESTful APIs expose endpoints for move submission, board state retrieval, analysis queries, and user statistics.
- Modular design allows pluggable AI backends, new scenario engines, or third-party chess services to be integrated or swapped as independent services.

4.2.3 Data/API Layer

- JSON serves as a universal serialization format for all API payloads, game states, and configuration data.
- WebSocket or HTTP endpoints encapsulate stateless and stateful interactions, supporting both turn-based and live (real-time) play.
- Persistence (DB or in-memory cache) is abstracted behind clear interfaces, allowing for easy swap or upgrade without affecting API clients.
- All scenario data (puzzles, training modules, variant rules) is stored and transferred as JSON objects, supporting easy import, versioning, and extension.

4.2.3 Integration and Interaction

- Upon user action, the frontend sends a JSON payload through the API layer to the Python backend.
- The backend leverages chess.js for fast legality checking, processes any AI computations, and returns an updated board state as JSON.
- Scenario modules may adjust the legal move set, AI heuristic, or challenge goals dynamically based on game phase or user configuration.
- State synchronization relies on push updates or client polling for low-latency experiences.

4.2.3 Rationale and Trade-offs

- Using Python for AI allows mature machine learning and chess AI libraries to be leveraged.

- chess.js in both backend and frontend assures consistency in rules enforcement, while React modularity empowers agile UI iteration.
- All game state and communication in JSON simplify integration testing, debugging, and future interoperability.
- Modular separation ensures most changes are local.

5. Detailed System Design

5.1 Classification

- Game Manager Module
- AI Engine Module
- UI Module
- State Management Module
- Scenario Loader Module
- Utilities Module (Validation, Logging)

5.2 Definition

- Game Manager Module: Orchestrates chess gameplay, managing turns, player input, and interactions between the AI and UI modules. It also maintains the core game state and communicates with scenario loading.
- AI Engine Module: Implements the chess AI, featuring both classic search (Minimax with Alpha-Beta pruning) and extensible machine learning-based evaluation. It determines AI moves and supports multiple difficulty levels and scenario-based constraints.
- UI Module: Handles all user interactions, board rendering, move input, and displays results. Designed for fast local feedback and easy resetting or testing of scenarios.
- State Management Module: Maintains game state in-memory, serializes positions as JSON/FEN, and supports undo/redo and scenario reloading.
- Scenario Loader Module: Loads and validates custom start positions, AI/game constraints, or training scenarios from local files (JSON or FEN). Supports rapid scenario switching for AI training or testing.
- Utilities Module: Provides chess move validation (chess.js or equivalent), logging, error handling, and configuration management services.

5.3 Constraints

- Designed for execution on typical local environments (laptops/desktops) running Python 3.8+ and supporting chess.js (JS validation/UI).
- Must perform state transitions and move validation atomically to prevent desynchronization, especially during rapid scenario testing or self-play loops.
- AI Engine must support switching evaluation functions to allow experimentation with both rule-based and machine learning-driven move selection.
- All components must permit stateless resets to support agile training and scenario exploration.
- Memory usage should remain efficient, supporting at least thousands of moves/states in RAM for analysis/training.

5.4 Resources

Hardware Resources:

- CPU processor: to execute the game logic, UI, AI computation moves
- RAM memory: To store the program instruction, game state, and PGN history
- Display: a monitor that displays the Chessboard UI inside a web browser
- Input: Mouse to drag and drop chess pieces and interact with the UI

Software Resources:

- Chessboard.js - UI chessboard visualization
- Chess.js - JavaScript Library for handling all game logic, with move validation and update game state.
- React Framework- Open-source JavaScript frameworks for building User Interfaces
- Flask API- python web application framework for server-side logic

Data Storage: all information will be transmitted to PGN/JSON format to store game state, game history (AI to generate game moves).

5.5 Interface/Exports

- Game Manager: API methods to start/restart game, apply player/AI move, load scenario, save state, and register callbacks.
- AI Engine: Interface for setting difficulty, choosing model/evaluation strategy, and returning a move for a given board state.

- UI: Functions for rendering, highlighting moves, capturing input, and displaying messages.
- State Management: Methods for fetching current state (JSON/FEN), history, undo/redo operations, and persisting game logs.
- Scenario Loader: Functions for validating/loading JSON/FEN scenario files, applying constraints dynamically.
- Utilities: Logging APIs, move validation, and configuration access exposed as shared services.

6. Glossary

Acronyms:

- PGN– Portable Game Notation (Chess game data format)
- API – Application Programming Interface
- CI/CD- Continuous Integration / Continuous Delivery
- JSON– JavaScript Object Notation (data format)
- React- JavaScript library used for building user interfaces
- FastAPI- Python Web Framework
- UI- User Interface
- JS- JavaScript a language used for frontend development

Our Domain Specific Terms:

- Move Validation: Is the process of making sure a move is legal and follows the rules of chess for both the user and AI.
- Game State: the current configuration of the chessboard, which includes chess pieces positioning and whose turn it is for the next move.

7. Bibliography

- FastAPI documentation: <https://fastapi.tiangolo.com>
- Python-chess library: <https://python-chess.readthedocs.io>
- React documentation: <https://react.dev>
- Chessboard.js: <https://chessboardjs.com>
- JSON: <https://www.json.org>

8. Basic UML Diagram of the System Architecture

